

CS-UY 3923/CS-GY 6813: Information Security & Privacy

Evan Johnson, Fall 2026, Lecture 2

Memory Corruption I: Control Flow Vulnerabilities

Logistics

- PA1 postponed to 9/22 at 5pm
 - For those who have finished: how was it?
- PA 2 released today
 - Will be due 5pm on 9/29 through gradescope
 - Gradescope link should become active in the next few days...
 - Start early!
- If you didn't sign the sign-up sheet last week, do so now

Last week

- How do we model security of software?
- Safety policies
- Attacker models
- Attack surfaces

Today: How safety policies are violated in practice (aka vulnerabilities)

What is a software vulnerability?

- A bug that allows your safety policy to be violated
- There are many types of vulnerabilities, but among the most important are those that violate “**control flow integrity**”
 - Translation: **lets attacker run their code on your computer**
- Typically, these involve violating *assumptions* of the programming language or its run-time system

Most high-impact vulnerabilities are from memory unsafety

Google: ~70% of Chrome's bugs (2015–2020)

Microsoft: ~70% of Windows' bugs (2006–2018)

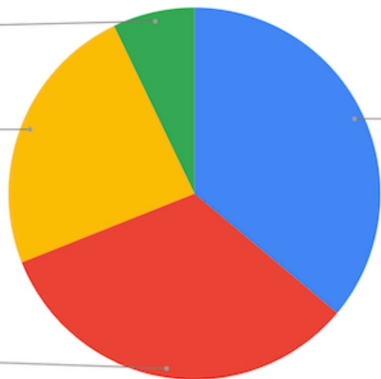
High+, impacting stable

Security-related assert
7.1%

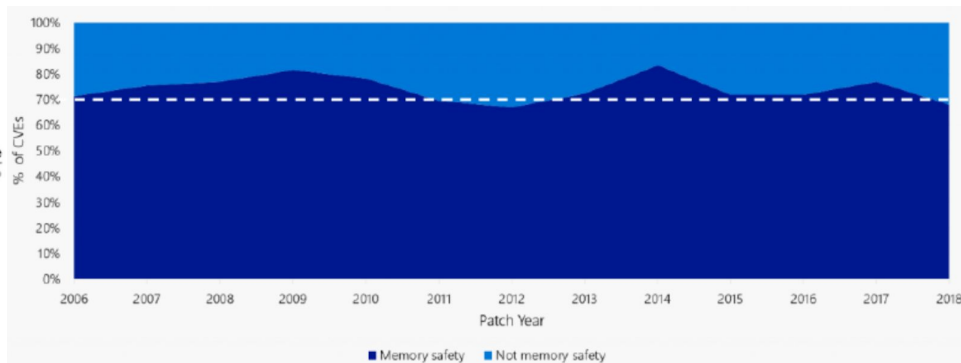
Other
23.9%

Other memory unsafety
32.9%

Use-after-free
36.1%



<https://www.chromium.org/Home/chromium-security/memory-safety/>



<https://msrc-blog.microsoft.com/2019/07/16/a-proactive-approach-to-more-secure-code/>

Starting exploits

- Today we dive into low level details of how exploits work
 - How can a remote attacker get **your** machine to execute **their** code?
- Our threat model
 - Victim code is **handling input** that comes across a security boundary
 - Examples:
 - Image viewer, word processor, web browser
 - Other examples?
- Simplest example: **buffer overflow**
 - Provide input that "overflows" the memory the program allocates for it

A quick peek ahead

- Today: Basic control flow vulnerabilities
- 9/22 (next week): Advanced (heap-based) vulnerabilities
- 9/29: Automated vuln-finding / tools of the trade
- After that: Defenses

The Plan

- Buffer overflows
- Format string vulnerabilities
- Integer overflows

The Plan

- Buffer overflows
- Format string vulnerabilities
- Integer overflows

Seem familiar?

```
[1] 311522 segmentation fault (core dumped) ./mycode
```

What does it mean?

Buffer Overflow

- A ***Buffer Overflow*** is occurs when a program writes data beyond the boundary of a buffer
- A basic idea that enables a broad range of possible attacks
 - Sometimes **a single byte** is all the attacker needs

Buffer Overflow: Why?

1. No automatic bounds checking in C/C++.
 - Developers need to get it right every single time.
2. Most C standard library functions don't check bounds.
 - E.g., string manipulation functions like `gets()`, `strcpy()`, and `strcat()` all **write to the destination buffer until they encounter a terminating `'\0'` byte** in the input.
 - Whoever is providing the input controls how much gets written

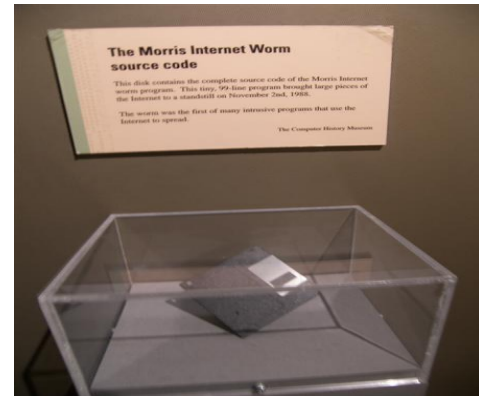
Example 1: fingerd

- Spot the vulnerability
- What does `gets()` do?
 - How many characters does it read in?
 - Who decides how much input to provide?
- How large is `line[]`?
 - Implicit assumption about input length

```
1 main(argc, argv)
2     char *argv[];
3 {
4     register char *sp;
5     char line[512];
6     struct sockaddr_in sin;
7     int i, p[2], pid, status;
8     FILE *fp;
9     char *av[4];
10
11     i = sizeof (sin);
12     if (getpeername(0, &sin, &i) < 0)
13         fatal(argv[0], "getpeername");
14     line[0] = '\0';
15     gets(line);
16     //...
17     return(0);
18 }
```

Morris Internet worm

- fingerd bug was one of several exploited by the “Morris worm”
 - Named after its author, then Cornell grad student, Robert Morris Jr.
 - Replicated itself – one of first Internet worms
- Effectively shut down much of the Internet in 1988
- Led to first conviction under the new US Computer Fraud and Abuse Act (CFAA)
- 37 years ago... surely not still a problem...



CVE-2025-0999 Detail

Description

Heap buffer overflow in V8 in Google Chrome prior to 133.0.6943.126 allowed a remote attacker to craft a HTML page. (Chromium security severity: High)

RICOH
imagine. change.

Search...



Change country

Business Solutions

Products & Services

Support

News & Events

About us

Insights

Home > News & Events > News > Ricoh has ident...ountermeasures.

11.07.2024

Specific Ricoh MFP and Printer Products - Buffer overflow vulnerability (CVE-2024-39927)

Specific Ricoh MFP and Printer Products - Buffer overflow vulnerability (CVE-2024-39927).



The Company ▾

Firmware Expertise ▾

Market Segments ▾

Press + Events

Contact ▾

Phoenix Technologies Buffer Overflow Vulnerability in TPM Configuration

📅 May 14, 2024

Phoenix was notified about an unsafe UEFI variable handling vulnerability in the TPM configuration for some platforms potentially leading to a buffer overflow.

Tracked under **CVE-2024-0762**, this vulnerability affects devices using Phoenix SecureCore firmware running on select Intel processor families including: AlderLake, CoffeeLake, CometLake, IceLake, JasperLake, KabyLake, MeteorLake, RaptorLake, RocketLake, and TigerLake.

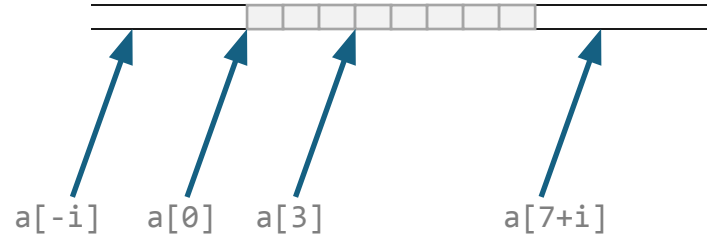
Mitigations for **CVE-2024-0762** were made available in April of 2024.

Ok, sure but...

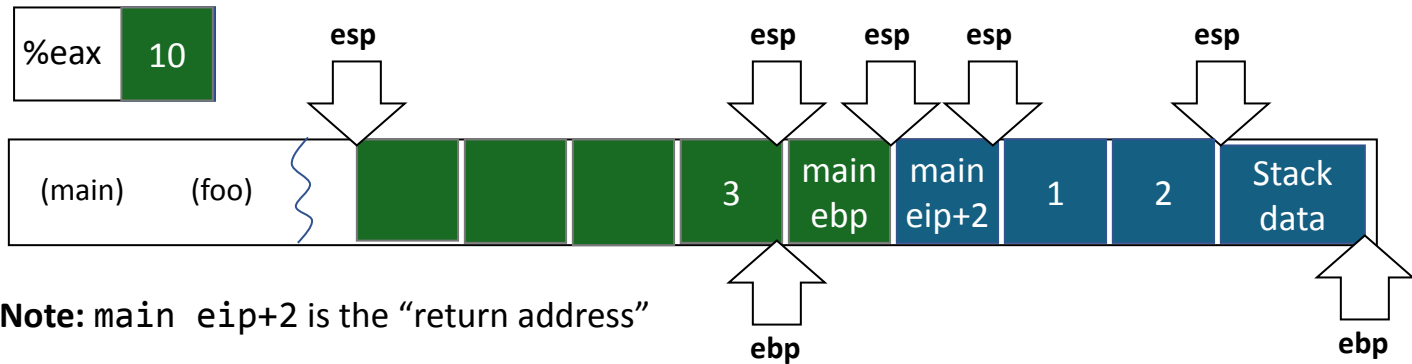
- I believe you can **crash** the computer with input that is too long, but... why does overflowing a buffer let you **take over the machine**?!?
- That seems crazy no?

Review #1: how do C arrays work?

- What's the abstraction?
- What's the reality?
 - Arrays exist in memory next to other variables (and compiler-inserted metadata)
 - Overwriting beyond array can overwrite that data



Review #2: how function calls work?



```
main:
eip → ...
eip → subl    $8, %esp
      movl    $2, 4(%esp)
      movl    $1, (%esp)
eip → call    foo
eip → addl    $8, %esp
      ...
```

```
foo:
eip → pushl   %ebp
      movl    %esp, %ebp
      subl    $16, %esp
eip → movl    $3, -4(%ebp)
eip → movl    8(%ebp), %eax
eip → addl    $9, %eax
eip → leave
eip → ret
```

Back to buffer overflows...

- So... consider this simple program ->
- It takes input from the the command line and then prints it followed by "is nice\n"
- How long can your input be?

```
#include <stdio.h>
#include <string.h>

int main(int argc, char**argv) {
    char nice[] = "is nice";
    char name[8];
    strcpy(name, argv[1]);
    printf("%s %s\n", name, nice);
    return 0;
}
```

Back to buffer overflows...

- So... consider this simple program ->
- It takes input from the the command line and then prints it followed by "is nice\n"
- How long can your input be?
- What happens if its longer?
Where does it go?

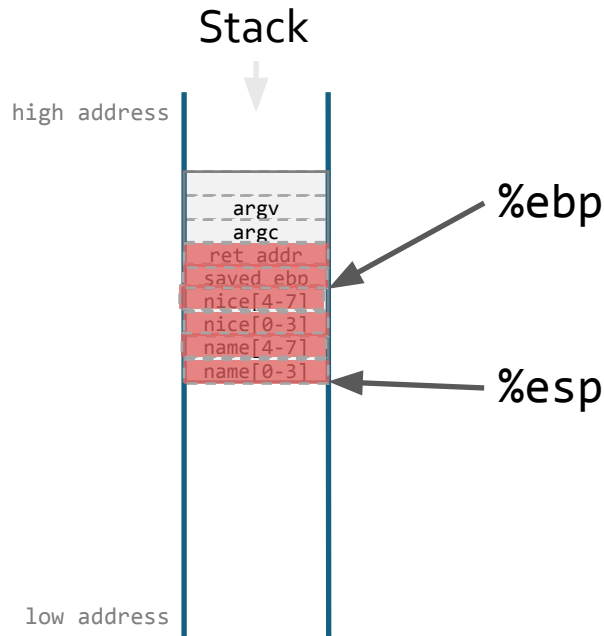
```
#include <stdio.h>
#include <string.h>

int main(int argc, char**argv) {
    char nice[] = "is nice";
    char name[8];
    strcpy(name, argv[1]);
    printf("%s %s\n", name, nice);
    return 0;
}
```

Smashing The Stack

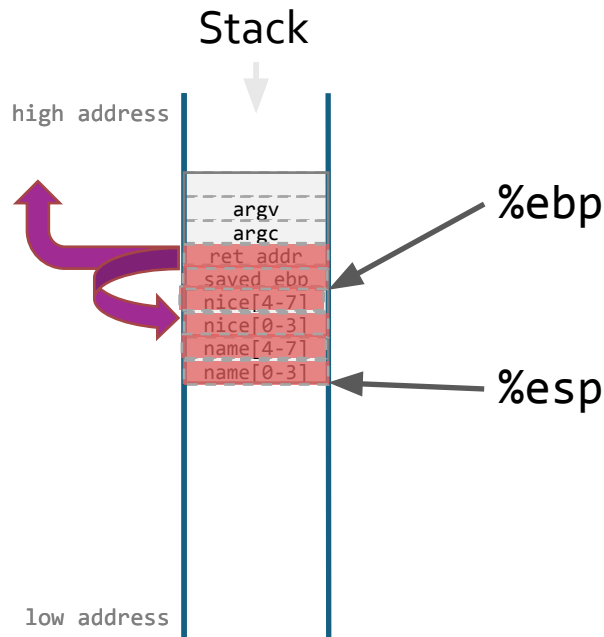
```
#include <stdio.h>
#include <string.h>

int main(int argc, char**argv) {
    char nice[] = "is nice";
    char name[8];
    strcpy(name, argv[1]);
    printf("%s %s\n",name,nice);
    return 0;
}
```



Smashing The Stack: control data

- The big one: the **return address**
 - On function return, control is transferred to an attacker-chosen address
- **Arbitrary code execution**
 - Attacker can re-direct to **their own code**
 - Shellcode! (coming up)
 - Reminder: there's **nothing** that distinguishes data from code
 - All data will be interpreted as code if the processor tries to transfer control there
- **Game over**



Shellcode

- What to do after we figure out how to gain control of the instruction pointer?
- Ideally, redirect to our own code!
- But what should that code be?
- Spawning a shell would provide us with full privileges of the victim process
 - Hence, “***shellcode***”

Shellcode

- How to spawn a shell?
- *“The exec family of functions shall replace the current process image with a new process image. The new image shall be constructed from a regular, executable file called the new process image file.”*
- Just need to call `execve` with the right arguments
 - `execve(“/bin/sh”, argv, NULL)`

Writing Shellcode

- Write shellcode in C
- Compile and run in debugger to review object code
- Adjust references to strings, etc.

```
void main() {  
    char *name[2];  
  
    name[0] = "/bin/sh";  
    name[1] = NULL;  
    execve(name[0], name, NULL);  
}
```

Aleph One's tips to writing Shellcode

- Test shellcode standalone
 - Encode shellcode into a data buffer
 - Set the return address on the stack to point to your shellcode

Aleph One's tips to writing Shellcode

- Test shellcode standalone
 - Encode shellcode into a data buffer
 - Set the return address on the stack to point to your shellcode
- Eliminating `0x00` from the shellcode (why?)
 - Find alternate instruction representations

Aleph One's tips to writing Shellcode

- Test shellcode standalone
 - Encode shellcode into a data buffer
 - Set the return address on the stack to point to your shellcode
- Eliminating `0x00` from the shellcode (why?)
 - Find alternate instruction representations
- Use a NOP sled (why?)

Aleph One's tips to writing Shellcode

- Test shellcode standalone
 - Encode shellcode into a data buffer
 - Set the return address on the stack to point to your shellcode
- Eliminating `0x00` from the shellcode (why?)
 - Find alternate instruction representations
- Use a NOP sled (why?)
 - Relaxes constraints on guessing the exact location of the shellcode to put into the overwritten return address
 - Jump to somewhere in NOP sled and slide down to shellcode

Remote Shellcode

- Works well for local attacks
 - When the victim is another process on the same machine
- What about remote attacks?
- Similar concept, just a few more system calls in the shellcode
 - Reverse
 - Connect back to your malicious server via network and present a remote shell
 - Bind
 - Open a network port and wait for connections, present shell
 - Reuse
 - Re-use existing network connection

Remote Shellcode

- Works well for local attacks
 - When the victim is another process on the same machine
- What about remote attacks?

Ok, but how do we find overflows?

- Manual evaluation (pattern-matching)
- Automated bug finding (fuzzing, static analysis, AI)
 - We'll cover this in 2 weeks

Buffer Overflow Pattern 1: missing check

- Missing Check
 - No test to make sure memory writes stay within bounds
- Example
 - fingerd

```
1 main(argc, argv)
2     char *argv[];
3 {
4     register char *sp;
5     char line[512];
6     struct sockaddr_in sin;
7     int i, p[2], pid, status;
8     FILE *fp;
9     char *av[4];
10
11     i = sizeof (sin);
12     if (getpeername(0, &sin, &i) < 0)
13         fatal(argv[0], "getpeername");
14     line[0] = '\0';
15     gets(line);
16     //...
17     return(0);
18 }
```

Buffer Overflow Pattern 2: avoidable check

- The test to check memory write is safe can be bypassed
- Example
 - libpng png_handle_tRNS()
 - 2004

```
356     if (png_ptr->color_type == PNG_COLOR_TYPE_PALETTE)
357     {
358         if (!(png_ptr->mode & PNG_HAVE_PLTE))
359         {
360             /* Should be an error, but we can cope with it */
361             png_warning(png_ptr, "Missing PLTE before tRNS");
362         }
363         else if (length > png_ptr->num_palette)
364         {
365             png_warning(png_ptr, "Incorrect tRNS chunk length");
366             png_crc_skip(png_ptr, length);
367             return;
368         }
```

Buffer Overflow Pattern 3: late check

- There is a test to check memory write is safe, but it is placed after the offending operation

```
1 #define BUFLen 20
2
3 void foo(char *s)
4 {
5     char buf[BUFLen];
6
7     strcpy(buf, s);
8     if(strlen(buf) >= BUFLen)
9     {
10         //handle error
11     }
12 }
```

Buffer Overflow Pattern 4: wrong check

- The test to make sure memory writes stay within intended bounds is wrong.
 - Look for complicated runtime arithmetic in length checks.
 - Stay tuned for integer errors...
 - Is NULL terminator accounted for?
 - If you see non-trivial arithmetic operations inside a length check, assume something is wrong!
- Example
 - OpenBSD realpath()
 - August 2003

```
124      /*
125       * Join the two strings together, ensuring that the right thing
126       * happens if the last component is empty, or the dirname is root.
127       */
128      if (resolved[0] == '/' && resolved[1] == '\\0')
129          rootd = 1;
130      else
131          rootd = 0;
132
133      if (*wbuf) {
134          if (strlen(resolved) + strlen(wbuf) + rootd + 1 > MAXPATHLEN) {
135              errno = ENAMETOOLONG;
136              goto err1;
137          }
138          if (rootd == 0)
139              (void)strcat(resolved, "/");
140          (void)strcat(resolved, wbuf);
141      }
```

Addressing Buffer Overflows

- The best way to deal with any bug is to prevent it by design.
 - Use memory-safe languages (e.g., Rust, Java)
 - Train the developers to write secure code and provide them with tools that make it easier to do so.
- Language choice might not be an option (it often isn't) and people still make mistakes. So, we must also proactively look for these bugs and fix them.
 - Manual code reviews, static analysis, adversarial testing, etc.
- Failing all of the above, use compiler-based countermeasures to make remaining bugs harder to exploit. (in a later lecture)

Review: Buffer Overflows

- An attacker can direct the execution of your program by manipulating input data it acts on.
- Assume input can be malicious. **Always** validate lengths and bounds before accessing arrays.
- If attacker can overwrite program memory, they may be able to run their own malicious code

Review: Finding buffer overflows

- Thinking like an attacker:
 - Missing Check
 - Does the code perform bounds checking on memory access?
 - Avoidable Check
 - Is the test invoked along every path leading up to actual access?
 - Wrong Check
 - Is the test correct? Can the test itself be attacked?
- Generic input validation patterns
 - Applicable beyond just buffer overflows

The Plan

- Buffer overflows
- Format string vulnerabilities
- Integer overflows

Anything wrong with this program?

```
#include <stdio.h>
```

```
int main(int argc, char** argv) {  
    char msg[80];  
    fgets(msg, 80, stdin);  
    printf(msg);  
}
```

Format Strings in C

Proper use of printf format string:

```
int foo = 1234;
```

```
printf("foo = %d in decimal, %X in hex", foo, foo);
```

This will print: `foo = 1234 in decimal, 4D2 in hex`

Format Strings in C

Proper use of printf format string:

```
int foo = 1234;  
printf("foo = %d in decimal, %X in hex", foo, foo);
```

This will print: `foo = 1234 in decimal, 4D2 in hex`

Sloppy use of printf format string:

```
char buf[14] = "Hello, world!";  
printf(buf);  
// should've used printf("%s", buf);
```

Question: What happens if buffer contains format symbols starting with % ???

Answer: They will be interpreted as arguments!

Consider this program again

```
#include <stdio.h>
```

```
int main(int argc, char** argv) {  
    char msg[80];  
    fgets(msg, 80, stdin);  
    printf(msg);  
}
```

I pass in “%d”

It prints 135148320

(why?)

What are other formats we can use?

Conversion Specifier	Explanation	Expected Argument Type									
		Length Modifier→									
		hh	h	none	l	ll	j	z	t	L	
		Only available since C99→									
%	Writes literal %. The full conversion specification must be %%. 	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A
c	Writes a single character . - The argument is first converted to <code>unsigned char</code>. - If the <code>l</code> modifier is used, the argument is first converted to a character string as if by <code>%ls</code> with a <code>wchar_t[2]</code> argument.	N/A	N/A	<code>int</code>	<code>wint_t</code>	N/A	N/A	N/A	N/A	N/A	N/A
s	Writes a character string . - The argument must be a pointer to the initial element of an array of characters. <i>Precision</i> specifies the maximum number of bytes to be written. If <i>Precision</i> is not specified, writes every byte up to and not including the first null terminator. - If the <code>l</code> specifier is used, the argument must be a pointer to the initial element of an array of <code>wchar_t</code>, which is converted to char array as if by a call to <code>wcrtomb</code> with zero-initialized conversion state.	N/A	N/A	<code>char*</code>	<code>wchar_t*</code>	N/A	N/A	N/A	N/A	N/A	N/A
d i	Converts a signed integer into decimal representation <code>[-]dddd</code> . <i>Precision</i> specifies the minimum number of digits to appear. The default precision is <code>1</code>. - If both the converted value and the precision are <code>0</code> the conversion results in no characters. - For the <code>z</code> modifier, the expected argument type is the signed version of <code>size_t</code>.	<code>signed char</code>	<code>short</code>	<code>int</code>	<code>long</code>	<code>long long</code>	<code>intmax_t</code>	<code>*</code>	<code>ptrdiff_t</code>		N/A

<https://en.cppreference.com/w/c/io/fprintf>

What are other formats we can use?

<https://en.cppreference.com/w/c/io/fprintf>

n	Returns the number of characters written so far by this call to the function.	signed char*	short*	int*	long*	long long*	intmax_t*	*	ptrdiff_t*	N/A
	- The result is <i>written</i> to the value pointed to by the argument. - The specification may not contain any <i>flag</i>, <i>field width</i>, or <i>precision</i>. - For the z modifier, the expected argument type is S*, where S is the signed version of <code>size_t</code>.									
p	Writes an implementation defined character sequence defining a pointer .	N/A	N/A	void*	N/A	N/A	N/A	N/A	N/A	N/A

Why does this exist?

Maybe to align text in old programs?

Writing Stack with Format Strings

`%n` : write the number of characters that have been printed

```
printf("We have written: %n",&myVar);
```

- Argument of printf is interpreted as destination address
- This writes 17 into myVar (message has 17 characters)

Writing Stack with Format Strings

`%n` : write the number of characters that have been printed

```
printf("We have written: %n",&myVar);
```

- Argument of `printf` is interpreted as destination address
- This writes `17` into `myVar` (message has 17 characters)

What if `printf` does not have an argument?

```
fgets(msg, 80, stdin);
```

```
printf(msg);
```

Writing Stack with Format Strings

`%n` : write the number of characters that have been printed

```
printf("We have written: %n",&myVar);
```

- Argument of `printf` is interpreted as destination address
- This writes `17` into `myVar` (message has 17 characters)

What if `printf` does not have an argument?

```
fgets(msg, 80, stdin);
```

```
printf(msg);
```

- This treats random data on the stack as a pointer, and we will write `17` to it

Key problem

- Format string describes a “command” for the runtime (read %d, write %n)
- If attackers “inject” their own input, they can run their own commands
- Format string attacks aren’t too common, but command injections are alive and well:
 - SQL injection
 - Prompt injections

Integers and Integer overflow

What does this code print?

```
a = 100;  
b = 200;  
c = a+b;  
printf("%d %d %d\n",  
       (int)a,  
       (int)b,  
       (int)c);
```

Well, it depends on how `a`, `b`, and `c` are defined!

What does this code print?

```
int32_t a = 100;  
int32_t b = 200;  
int32_t c = a+b;  
printf("%d %d %d\n",  
       (int)a,  
       (int)b,  
       (int)c);
```



```
100  
200  
300
```

Well, it depends on how `a`, `b`, and `c` are defined!

What does this code print?

```
uint8_t a = 100;  
uint8_t b = 200;  
uint8_t c = a+b;  
printf("%d %d %d\n",  
       (int)a,  
       (int)b,  
       (int)c);
```



```
100  
200  
44
```

Well, it depends on how `a`, `b`, and `c` are defined!

What does this code print?

```
int8_t a = 100;  
int8_t b = 200;  
int8_t c = a+b;  
printf("%d %d %d\n",  
       (int)a,  
       (int)b,  
       (int)c);
```



```
100  
-56  
44
```

Well, it depends on how `a`, `b`, and `c` are defined!

An `int` is not an integer

- C defines fixed-width integer types (`int`, `long`, etc.) that do not always behave like Platonic integers from elementary school.
- Because of the fixed width, it is possible to overflow or wrap max number for the type used
 - Or underflow in case of negative numbers

Spot the vulnerability

```
my_type* foo(int n)
{
    my_type *ptr = malloc(n * sizeof(my_type));
    for(int i = 0; i < n; i++)
    {
        memset(&ptr[i], i, sizeof(my_type));
    }
    return ptr;
}
```

Integer Overflow/Underflow behavior

- Unsigned Overflow/Underflow: wraps.
 - As if arithmetic operation was performed modulo $2^{\text{(size of the type)}}$.
- Signed Overflow/Underflow: undefined in C.
 - But most implementations wrap.

Fixing int overflows idea 1: check result

```
my_type* foo(uint32_t a, uint32_t b)
{
    . . .
    if(a+b > UINT32_MAX)
    {
        . . .
    }
    return ptr;
}
```

Does this work?

Remember: $a+b == (a+b) \bmod \text{UINT32_MAX}$

Fixing int overflows idea 2: promote types

```
my_type* foo(uint32_t a, uint32_t b)
{
    . . .
    if((uint64_t)a + (uint64_t)b > UINT64_MAX)
    {
        . . .
    }
    return ptr;
}
```

Does this work?

Yes, but only if we have bigger ints available

Fixing int overflows idea 3: reverse check

```
my_type* foo(uint32_t a, uint32_t b)
{
    . . .
    if(UINT32_MAX - a < b)
    {
        . . .
    }
    return ptr;
}
```

Does this work?

Fixing int overflows idea 3: reverse check

- Unsigned overflow checks use the **complementary** operation to the one being checked
 - Subtraction to check for addition overflow
 - `if (UINT32_MAX - a < b)`
 - Division to check for multiplication overflow
 - `if ((0 != a) && (UINT32_MAX / a < b))`
- More complex for signed types
 - `if(((a>0) && (b>0) && (a > (INT32_MAX-b))) || ((a<0) && (b<0) && (a < (INT32_MIN-b))))`

Fixing int overflows idea 3: reverse check

- Unsigned overflow checks use the **complementary** operation to the one being checked
 - Subtraction to check for addition overflow
 - `if (UINT32_MAX - a < b)`
 - Division to check for multiplication overflow
 - `if ((0 != a) && (UINT32_MAX / a < b))`
- More complex for signed types
 - `if(((a>0) && (b>0) && (a > (INT32_MAX-b))) || ((a<0) && (b<0) && (a < (INT32_MIN-b))))`
- Do you think people do this? and correctly if so?

Fixing int overflows idea 3: better APIs

1. `a.trapping_add(b)` => throws exception on overflow
2. `a.saturating_add(b)` => returns max value (e.g, `UINT32_MAX`)
3. `a.check_add(b)` => returns `Option<int>` (which must be checked before use)
4. `a.wrapping_add(b)` => “normal” addition

That was a lot of work safely to add $a + b$!

That was a lot of work safely to add $a + b$!

But our integer troubles aren't over yet...

Integer Type Conversion

- Integer type conversions are another source of vulnerabilities
 - Whenever a value is changed from one type to another.
- How are values converted from one type to another? What happens to the bit??
 - Truncation
 - Zero-extension
 - Sign-extension

Integer Conversion 1: Truncation

- **Truncation** occurs when a value with a wider type is converted to a narrower type.
- When a value is truncated, its high-order bytes are removed so that it is the same width as the narrower type.

```
uint32_t j = 0xDEADBEEF;  
uint16_t i = j;  
// i == 0xBEEF
```

Integer Conversion 2: Zero-extension

- **Zero-extension** occurs when a value with a narrower, unsigned type is converted to a wider type.

```
uint16_t i = 0xBEEF;  
uint32_t j = i;  
// j == 0x0000BEEF
```

- When a value is zero-extended, it is widened so that it is the same width as the wider type.
 - The new bytes are zero.

Integer Conversion 3: Sign-extension

- **Sign-extension** occurs when a value with a narrower, signed type is converted to a wider type.
- When a value is sign-extended, it is widened so that it is the same width as the wider type.
 - If the sign bit of the original value is set, the new bytes are set.
 - If the sign bit of the original value is unset, the new bytes are unset.
- Sign-extension is used for signed types rather than zero-extension because it preserves the value.

```
int8_t i = -127; //      1000 0001
int8_t j = 127;  //      0111 1111
int16_t ki = i;
int16_t kj = j;
// ki == 1111 1111 1000 0001
// kj == 0000 0000 0111 1111
```

Integer Type Conversion

- The following two slides display reference charts that explain particular integer type conversions.
- **You have no obligation to memorize these**

Integer Type Conversion (for reference)

From	To	Method	Lost or Misinterpreted
unsigned char	signed char	Preserve bit pattern; high-order bit becomes sign bit	Misinterpreted
unsigned char	signed short int	Zero-extend	Safe
unsigned char	signed long int	Zero-extend	Safe
unsigned char	unsigned short int	Zero-extend	Safe
unsigned char	unsigned long int	Zero-extend	Safe
unsigned short int	signed char	Preserve low-order byte	Lost
unsigned short int	signed short int	Preserve bit pattern; high-order bit becomes sign bit	Misinterpreted
unsigned short int	signed long int	Zero-extend	Safe
unsigned short int	unsigned char	Preserve low-order byte	Lost
unsigned long int	signed char	Preserve low-order byte	Lost
unsigned long int	signed short int	Preserve low-order word	Lost
unsigned long int	signed long int	Preserve bit pattern; high-order bit becomes sign bit	Misinterpreted
unsigned long int	unsigned char	Preserve low-order byte	Lost
unsigned long int	unsigned short int	Preserve low-order word	Lost

Integer Type Conversion (for reference)

From	To	Method	Lost or Misinterpreted
signed char	short int	Sign-extend	Safe
signed char	long int	Sign-extend	Safe
signed char	unsigned char	Preserve bit pattern; high-order bit no longer sign bit	Misinterpreted
signed char	unsigned short int	Sign-extend to short; convert short to unsigned short	Lost
signed char	unsigned long int	Sign-extend to long; convert long to unsigned long	Lost
signed short int	char	Truncate to low-order byte	Lost
signed short int	long int	Sign-extend	Safe
signed short int	unsigned char	Truncate to low-order byte	Lost
signed short int	unsigned short int	Preserve bit pattern; high-order bit no longer sign bit	Misinterpreted
signed short int	unsigned long int	Sign extend to long; convert long to unsigned	Lost
signed long int	char	Truncate to low order byte	Lost
signed long int	short int	Truncate to low order bytes	Lost
signed long int	unsigned char	Truncate to low order byte	Lost
signed long int	unsigned short int	Truncate to low order bytes	Lost

Integer Conversions can be implicit

- Type conversion can be **explicit** or **implicit**
- Explicit type conversions use the casting syntax:
 - `int i = (int) 4.5;`
- Implicit type conversions:
 - `signed short i = 1;      // assignment conversion`
 - `if (i < j) { ... }      // comparison conversion`
 - `void function(int arg);
 function(5.3); // function argument conversion`

But what does this have to do with security?

Spot the vulnerability

```
struct sockaddr_in
{
    short    sin_family;
    u_short  sin_port; ← 16 bit unsigned
    //...
};
//...
sockaddr_in sockaddr;
32 bit signed → int port; // user input
//...
if(port < 1024 && !is_root)
    // handle error
else
    sockaddr.sin_port = port;
```

- Note: Ports <= 1024 are reserved for critical services

Spot the vulnerability

```
struct sockaddr_in
{
    short    sin_family;
    u_short  sin_port; ← 16 bit unsigned
    //...
};
//...
sockaddr_in sockaddr;
32 bit signed → int port; // user input
//...
if(port < 1024 && !is_root)
    // handle error
else
    sockaddr.sin_port = port;
```

- Note: Ports <= 1024 are reserved for critical services

Spot the vulnerability

```
struct sockaddr_in
{
    short    sin_family;
    u_short  sin_port; ← 16 bit unsigned
    //...
};
//...
sockaddr_in sockaddr;
32 bit signed → int port; // user input
//...
if(port < 1024 && !is_root)
    // handle error
else
    sockaddr.sin_port = port;
```

port = 65616



sin_port = 80
(http!)

$$65616 \% 2^{16} \\ = 80$$

- Note: Ports ≤ 1024 are reserved for critical services

Addressing integer overflow/conversions

- Strongly typed language
 - Most such problems go away in Java or Rust for example
- Runtime checking
 - gcc -ftrapv (trap on signed overflow on add, sub, mult)
- Static checking (range analysis)
 - Sarkar et al, “Flow-insensitive Static Analysis for Detecting Integer Anomalies in Programs”

Additional Resources

- *Memory Corruption Attacks: The Almost Complete History* by Haroon Meer, Black Hat USA 2010
 - <https://www.youtube.com/watch?v=stVz9rhTdQ8>
- *Code Injection in C and C++ : A Survey of Vulnerabilities and Countermeasures* by Yves Younan, Wouter Joosen, Frank Piessens
 - www.cs.kuleuven.be/publicaties/rapporten/cw/CW386.pdf
- *Exploiting Format String Vulnerabilities* by scut / team teso
 - <https://cs155.stanford.edu/papers/formatstring-1.2.pdf>

For next time

- Read *Memory Errors: The Past, the Present, and the Future* by Victor van der Veen, Nitish dutt-Sharma, Lorenzo Cavallaro, and Herbert Bos
- http://www.few.vu.nl/~herbertb/papers/memerrors_raid12.pdf